

PhytoPi-ESP32

SWE-410: Software Engineering

Milestone 5 — Initial Project Scope Completion

Multi-Sensor IoT Telemetry with MQTT and Grafana

Team: PhytoPi

Nolan Tuttle | Daniel Grijalva

Instructor: Professor Bill Hughes

Submission Date: April 26, 2026

Contents

Version and Team Information	2
Signatures	2
GitHub Repository.....	2
Video Presentation.....	3
I. Project Overview.....	3
II. System Architecture.....	3
ESP32-S3 (Sensor Node).....	3
Data Flow	3
Raspberry Pi (Broker + Visualization).....	3
III. Code Carry-Forward and Refinement Summary	4
IV. Sensor Integration Design.....	4
Thermistor (thermistor_wrapper)	4
Light Sensor (light_sensor_wrapper).....	5
Soil Moisture Sensor (soil_sensor_wrapper).....	5
V. MQTT Topic Design	5
VI. Raspberry Pi Live Data Display	6
VII. Build / Flash / Run Procedure	6
Prerequisites	6
Build.....	6
Flash and Monitor.....	6
Raspberry Pi — Start Broker and Grafana	7
Credentials (main.c).....	7
VIII. Testing and Verification.....	7
ESP32 Serial Monitor.....	7
Pi-Side MQTT Verification	7
Grafana Dashboard.....	7
Sensor Sanity Checks.....	8
IX. Issues Encountered and Resolutions	8
1. DHT11 Timing Sensitivity Under Active Wi-Fi.....	8
2. app_main Blocking on Event Group Wait.....	8
3. Capacitive Soil Sensor Output Polarity.....	8
4. Mosquitto Password File Ownership	8
X. Conclusion	8
References.....	9

Version and Team Information

Version	5.0
Revision Date	April 26, 2026
Team Members	Nolan Tuttle, Daniel Grijalva

Signatures

Nolan Tuttle

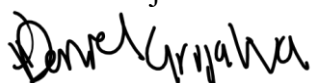
Nolan Tuttle



Signature / Date

Daniel Grijalva

Daniel Grijalva



Signature / Date

GitHub Repository

<https://github.com/nolantuttle/PhytoPi-ESP32>

Video Presentation

<https://youtu.be/8QHsvJfKrY>

I. Project Overview

Milestone 5 completes the initial scope of the PhytoPi-ESP32 project by integrating three analog sensors with an ESP32-S3, publishing live telemetry over MQTT to a Raspberry Pi broker, and displaying the data on a Grafana dashboard. The goal was to demonstrate a functional end-to-end IoT pipeline: from physical sensor readings through wireless transmission to live visualization. The project is part of a larger capstone effort (PhytoPi) focused on automated plant care. Milestone 5 establishes the embedded telemetry foundation that the final system will build on.

II. System Architecture

The system consists of two primary nodes: an ESP32-S3 acting as the sensor and publishing node, and a Raspberry Pi running the broker and visualization stack.

ESP32-S3 (Sensor Node)

Hardware target: ESP32-S3, flashed via ESP-IDF v5.2.6. The firmware is structured as follows:

```
app_main
├─ nvs_flash_init          (NVS required by Wi-Fi stack)
├─ WifiManagerInit        (STA mode, WPA2-PSK, event-driven reconnect)
├─ adc_oneshot_new_unit    (ADC1 shared handle)
├─ ThermistorWrapperInit   (ADC1, channel 4 / GPIO5)
├─ LightSensorWrapperInit  (ADC1, channel 1 / GPIO2)
├─ SoilSensorWrapperInit   (ADC1, channel 0 / GPIO1)
└─ xTaskCreate → PublishTelemetryTask (8 KB stack, priority 5)
```

Data Flow

[Sensors] → ADC1 → [Sensor Wrappers] → cJSON object → MqttManagerPublish → Mosquitto broker on Raspberry Pi @ 192.168.1.178:1883

Raspberry Pi (Broker + Visualization)

Mosquitto MQTT broker receives JSON payloads on topic `gcu/lab/env/telemetry`. The Grafana MQTT data source plugin subscribes to the same topic and renders live gauge panels for each sensor field.

III. Code Carry-Forward and Refinement Summary

The Milestone 5 codebase was built by carrying forward the working architecture from Milestone 3 (`esp32-dht11-mqtt-grafana-node`). The table below summarizes what was retained and what changed.

Component	Carry-Forward	Changes in Milestone 5
wifi_manager	Full logic	PascalCase naming; WPA2-PSK auth mode set explicitly
mqtt_manager	Full logic	Added <code>MqttManagerIsReady()</code> ; explicit username/password args; stack bumped to 8192
main.c structure	Task loop, NVS init	DHT11 replaced with three ADC sensors; cJSON payloads; all three sensor reads gated before publish

The Milestone 2 component stubs (`bme680_driver`, `soil_moisture`, etc.) were scaffold-only with no implementation and were not carried forward. Only the component directory pattern — one folder per sensor, each with a `CMakeLists.txt`, `.c`, and `.h` — was reused.

IV. Sensor Integration Design

All three sensors are analog and share a single `adc_oneshot_unit_handle_t` (ADC1), created once in `app_main` and passed by handle into each wrapper's `Init` function. Each wrapper stores the handle as a module-level static. All wrappers follow the same API pattern:

```
esp_err_t XxxWrapperInit(adc_oneshot_unit_handle_t adc1_handle);
esp_err_t XxxWrapperRead(float *output);
```

Thermistor (`thermistor_wrapper`)

- GPIO5 → ADC1 channel 4
- Hardware: 10 kΩ NTC thermistor, B=3950 K, 10 kΩ series resistor to 3.3 V (voltage divider, sensor to GND)
- Math: voltage divider → `r_therm` → Steinhart-Hart β formula → °C

- Attenuation: ADC_ATTEN_DB_12 (full-scale ~3100 mV on S3)
- Returns: float temperatureC

Light Sensor (light_sensor_wrapper)

- GPIO2 → ADC1 channel 1
- Hardware: photoresistor with analog voltage output
- Math: $\text{raw} / 4095 \times 100 \rightarrow \text{percentage}$
- Returns: float lightPct (0–100%)

Soil Moisture Sensor (soil_sensor_wrapper)

- GPIO1 → ADC1 channel 0
- Hardware: capacitive sensor — output voltage is inverted (higher voltage = drier)
- Math: $(\text{DRY_RAW} - \text{raw}) / (\text{DRY_RAW} - \text{WET_RAW}) \times 100$, clamped 0–100%
- Calibration: SOIL_DRY_RAW = 350, SOIL_WET_RAW = 280 (require per-unit calibration)
- Returns: float moisturePct (0–100%)

V. MQTT Topic Design

Broker: Mosquitto on Raspberry Pi, 192.168.1.178:1883, authenticated.

Topic: gcu/lab/env/telemetry

Payload format (JSON, single line, published every 5 seconds):

```
{"device":"esp32-01","temperature_c":24.3,"light_pct":61.2,"soil_moisture_pct":45.0}
```

Key	Type	Unit
device	string	node identifier
temperature_c	number	°C
light_pct	number	0–100%
soil_moisture_pct	number	0–100%

QoS: 1 (at least once), Retain: 0. All three sensor reads must return ESP_OK before a payload is assembled. Any failure logs a warning and skips that cycle.

Subscribe command (Pi terminal):

```
mosquitto_sub -h 127.0.0.1 -t 'gcu/lab/env/telemetry' -u nolan -P rootroot -v
```

VI. Raspberry Pi Live Data Display

Grafana is installed on the Raspberry Pi Zero 2 W and accessible via browser at <http://<pi-ip>:3000>. The MQTT data source plugin connects directly to the local Mosquitto broker on port 1883 using authenticated credentials. Three gauge panels display live values for temperature, light, and soil moisture, each labeled with appropriate units and updating in real time.

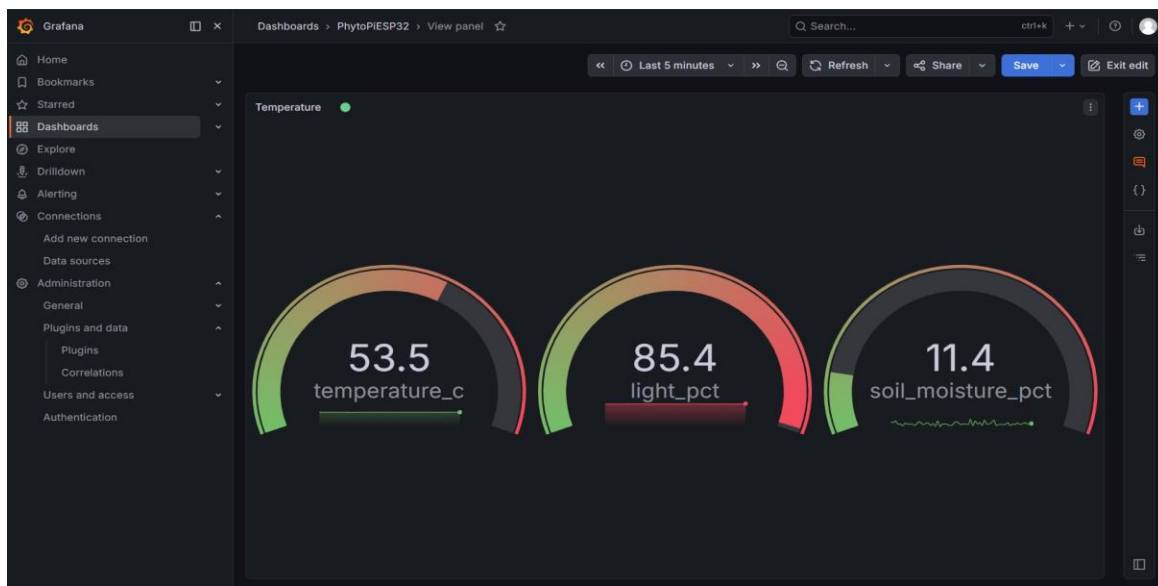


Figure 1: Grafana dashboard showing live temperature_c, light_pct, and soil_moisture_pct gauge panels.

VII. Build / Flash / Run Procedure

Prerequisites

- ESP-IDF v5.2.6 sourced (esp-idf/export.sh or equivalent)
- Target chip: ESP32-S3

Build

```
cd esp32-multisensor-mqtt-rpi-node
idf.py set-target esp32s3      # first time only
idf.py build
```

Flash and Monitor

```
idf.py -p COM4 flash monitor # Windows; use /dev/ttyUSB0 on Linux
```

Raspberry Pi — Start Broker and Grafana

```
sudo systemctl start mosquitto
sudo systemctl start grafana-server
```

Credentials (main.c)

```
#define WIFI_SSID      "YOUR_SSID"
#define WIFI_PASSWORD  "YOUR_PASSWORD"
#define MQTT_BROKER_URI "mqtt://192.168.1.178:1883"
#define MQTT_USERNAME  "nolan"
#define MQTT_TOPIC      "gcu/lab/env/telemetry"
```

VIII. Testing and Verification

The system was verified end-to-end across three layers.

ESP32 Serial Monitor

The idf.py monitor output confirms Wi-Fi association, IP assignment, and repeated JSON publish cycles. The mqtt_manager logs a successful broker connection handshake.



```
PS C:\Users\nolan\Documents\PhytoPiESP32\esp32-dht11-mqtt-grafana-node> & 'C:\Espressif\tools\python\v5.2.6\venv\Scripts\python.exe' 'C:\esp\v5.2.6\esp-idf\tools\idf_monitor.py' -p COM7 -b 115200 --toolchain-prefix xtensa-esp32s3-elf- --make "'C:\Espressif\tools\python\v5.2.6\venv\Scripts\python.exe'" 'C:\esp\v5.2.6\esp-idf\tools\idf.py' --target esp32s3 'c:\Users\nolan\Documents\PhytoPiESP32\esp32-dht11-mqtt-grafana-node\build\esp32-dht11-mqtt-grafana-node.elf'
I (824666) SMART_LAB_NODE: Published msg_id=15237: {"device":"esp32-01","temperature_c":53.3179931640625,"light_pct":84.3223419189453,"soil_moisture_pct":0}
I (824676) mqtt_manager: MQTT message published successfully.
I (829666) SMART_LAB_NODE: Published msg_id=9480: {"device":"esp32-01","temperature_c":53.453704833984375,"light_pct":84.2002487182617,"soil_moisture_pct":0}
I (829676) mqtt_manager: MQTT message published successfully.
I (834666) SMART_LAB_NODE: Published msg_id=19396: {"device":"esp32-01","temperature_c":53.38580322265625,"light_pct":84.2246627807617,"soil_moisture_pct":4.2857146263122559}
```

Figure 2: ESP-IDF serial monitor showing live published JSON payloads with temperature, light, and soil moisture values.

Pi-Side MQTT Verification

mosquitto_sub was run on the Pi terminal and confirmed incoming payloads for all three sensor fields.

```
mosquitto_sub -h 127.0.0.1 -t 'gcu/lab/env/telemetry' -u nolan -P rootroot -v
```


Grafana Dashboard

The Grafana MQTT data source was configured and confirmed live with three gauge panels actively updating, as shown in Figure 1.

Sensor Sanity Checks

- Thermistor: returns ESP_FAIL on open circuit or rail-clamp ADC reading
- Light: 0% = covered/dark, 100% = full illumination
- Soil: manual dry/wet calibration applied; values clamped 0–100%

IX. Issues Encountered and Resolutions

1. DHT11 Timing Sensitivity Under Active Wi-Fi

The DHT11 one-wire protocol is timing-critical and failed reliably when the Wi-Fi radio was active due to interrupt latency. Resolution: replaced with three ADC-based sensors (thermistor, capacitive soil, photoresistor). ADC oneshot reads are not timing-sensitive and tolerate Wi-Fi interrupts without failure.

2. app_main Blocking on Event Group Wait

The original firmware blocked app_main directly on xEventGroupWaitBits waiting for Wi-Fi, which starved the FreeRTOS scheduler and caused an RTC watchdog reset. Resolution: moved the wait, MQTT init, and publish loop into a dedicated FreeRTOS task (PublishTelemetryTask) with an 8 KB stack. app_main now returns immediately after creating the task.

3. Capacitive Soil Sensor Output Polarity

Capacitive sensors output higher voltage when dry, the opposite of intuitive expectation. Resolution: inverted math in soil_sensor_wrapper.c: $(\text{DRY_RAW} - \text{raw}) / (\text{DRY_RAW} - \text{WET_RAW})$. Additionally, the sensor required 5V for a usable ADC range.

4. Mosquitto Password File Ownership

Mosquitto failed to start with exit-code 13 (permission denied) when the passwd file was owned by root after recreation. Resolution: `sudo chown mosquitto:mosquitto /etc/mosquitto/passwd` and `chmod 640` applied after every passwd file write.

X. Conclusion

Milestone 5 delivered a working end-to-end IoT telemetry system. The ESP32-S3 reads three analog sensors, serializes the data as JSON, and publishes it over MQTT to a Raspberry Pi

broker every five seconds. Grafana displays the live values on a dashboard accessible from any browser on the local network.

The biggest technical lessons from this milestone were around FreeRTOS task structure — specifically that blocking `app_main` is not safe — and the importance of ADC handle ownership across multiple sensor components. The debugging process also reinforced systematic thinking: isolating each layer (WiFi, MQTT, broker auth, sensor wiring) before moving to the next.

This milestone establishes the sensor telemetry foundation for the final PhytoPi system, which will add actuator control (pump, fan, lighting) driven by the incoming sensor data.

References

- [1] Espressif Systems, "ESP-IDF Programming Guide," v5.2.6. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/v5.2.6/>
- [2] Eclipse Foundation, "Mosquitto MQTT Broker Documentation." [Online]. Available: <https://mosquitto.org/documentation/>
- [3] Grafana Labs, "Install Grafana on Raspberry Pi," Mar. 2026. [Online]. Available: <https://grafana.com/tutorials/install-grafana-on-raspberry-pi/>
- [4] DaveGamble, "cJSON — Ultralightweight JSON parser in ANSI C." [Online]. Available: <https://github.com/DaveGamble/cJSON>